

使用 Skill 开发 Flash Attention AI 算子的完整流程

基于 Codex 会话记录 (2026-03-10 ~ 2026-03-31 , 同一连续会话) 梳理
本文聚焦算子开发实施过程, 包括 Skill 如何指导开发、经验如何反哺 Skill

- 1. 全景时间线
- 2. Phase 1-2: flash_attention_simple — Skill 的首次实战 (3/10 ~ 3/12)
 - 2.1 用户指令
 - 2.2 Skill 如何指导开发
 - 2.3 开发中即时沉淀的经验 (反哺 Skill)
- 3. Phase 3: flash_attention_simple 板端优化 (3/13)
 - 3.1 优化迭代时间线
 - 3.2 Simple 版本的 Tile 配置
 - 3.3 Simple 版本沉淀的优化经验 (反哺 Skill)
- 4. Phase 4: Skill 体系完善 (3/16 ~ 3/23)
- 5. Phase 5-6: flash_attention_ai 开发
 - 5.1 用户指令
 - 5.2 方案包创建
 - 5.3 Skill 经验的应用
 - 5.4 FA_AI 的架构设计
 - 5.5 用户驱动的设计决策
 - 5.6 每一轮优化的详细分析: 实现 → Skill 触发点 → 性能结果
 - 5.6.0 测试配置与公式定义
 - Part A: flash_attention_simple 8 轮优化
 - Round 1 — 在线 Softmax one-pass 快路径
 - Round 2 — 首次完整 cmodel→板端→建模 闭环
 - Round 3 — 满块快路径: 消除冗余 fill
 - Round 4 — 消除冗余 cast: 在线 Softmax 路径精简
 - Round 5 — tile_n 128→256 扩展
 - Round 6 — single_k_tile 快路径 + 启发式调优
 - Round 7 — K/V 双缓冲 + pipeline
 - Round 8 — 失败实验 + 回退稳定
 - Simple 阶段汇总 (测试 shape: (1,8,2048,2048,64,64))
 - Part B: flash_attention_ai 17 步优化
 - Step 0 — 全新代码库创建
 - Step 1 — FA_AI 首次板端: MFU 12.94%
 - Step 2 — 动态 Tiling + tile_m 64→128: MFU 25.46%
 - Step 3 — Causal 快路径 + pipeline 重构: MFU 29.31%
 - Step 4 — Scratch buffer 移入 pipeline 循环: MFU 34.74%
 - Step 5 — Multicore 基线重测
 - Step 6 — choose_fa_tiles 重写 + V7.1 cycle 模型
 - Step 7 — wide_n 实验失败
 - Step 8 — Profile 对比分析: AI vs 手写的结构差距
 - Step 9 — 手写 multicore 三大结构优势识别
 - Step 10 — causal_widem 自动启用: MFU 38.46%
 - Step 11 — 多种实验均无效
 - Step 12 — AI 首次超越手写 causal: MFU 45.39%
 - Step 13 — Explicit mask 优化: host 缓存 + kernel 复用
 - Step 14 — MaskAnalysis: causal mask 语义降级到快路径: MFU ~46.3%
 - Step 15 — 动态 tile_bh 解决 LMEM 溢出
 - Step 16 — mask_window_headpack + 加权调度
 - Step 17 — 健康板卡最终测量: MFU 62.96%
 - 5.6.1 MFU 优化轨迹总图
 - 5.6.2 Skill 触发点频率统计
 - 5.6.3 性能对比总表
 - 5.7 衍生 Skill 的级联调用
 - 5.8 开发过程中新增的经验 (反哺 Skill)
- 6. Agent 遗忘与指令漂移分析
 - 6.1 会话规模
 - 6.2 观察到的遗忘/漂移现象
 - 现象 1: .inc 决策遗忘
 - 现象 2: 经验文档引用频率下降
 - 现象 3: HelloAGENTS Shell 格式松弛
 - 现象 4: Skill 验收闸门弱化
 - 6.3 遗忘模型

- 6.4 关键洞察
- 7. Skill 辅助算子开发的优势总结
 - 7.1 与纯手写开发的对比
 - 7.2 Skill 体系的核心价值
- 8. 当前不足与后续优化方向
 - 8.1 已识别的不足
 - 8.2 Skill 后续优化方向
- 附录
 - A. 性能数据来源
 - B. 板端测试命令
 - C. 关键文件路径
 - D. 会话记录位置

1. 全景时间线

整个开发过程发生在一个超长连续 Codex 会话 (3/10 启动，持续至 3/31)，记录 54000 行 JSONL。包含两个算子版本的完整生命周期：

	3/10	3/11-12	3/13	3/16-23	3/24	3/25-31
Phase1	Phase2	Phase3	Phase4	Phase5	Phase6	
Skill	Simple	Simple	Skill	FA_AI	FA_AI	
+Simple	+	opt1-opt12	Profile	60%MFU	62.96%	
		(3/13)				
MFU:	~0%	~28%44%	~10%	62.96%		

2. Phase 1-2: flash_attention_simple — Skill 的首次实战 (3/10 ~ 3/12)

2.1 用户指令

"仅根据 skill 的辅助，开发一个 flash attention 算子"
 "以越简单实现越好为主，最好可以一次性实现一个可跑通的版本"
 "确认开始，注意不要参考原有算子，直接当作是新算子来开发"

2.2 Skill 如何指导开发

Skill 的 16 阶段流水线在此次开发中首次完整走通：

	PPL	#1
	Shape [BH,M,1,K]	
Python Reference	Numpy	
PPL	flash_attention_simple.pl	#2-4
	MHA: shape, mask/causal	
.inc	(Simple)	
Torch-TPU	FlashAttentionSimple.cpp	#5
	flash_attention_simple.py	
CModel	ppl_compile.py --gen_ref	#6-8
	: "npz compare PASSED"	
Wheel	rebuild_TPU1686 new_clean new_build	#9-11
	: unused-function warning (-Werror)	
	Docker torch_tpu_py312 172.28.143.24	#12-15
Benchmark	shape	
-		

2.3 开发中即时沉淀的经验 (反哺 Skill)

experience_workflow.md 在 3/10-3/12 期间积累的关键经验：

```
#4 PPL USING_PPLUSE_PPL=1
#5 CModel PPL_RUNTIME_PATH, PPL_PROJECT_ROOT, LD_LIBRARY_PATH
#6 torch_tpu_py312
#9 : source envsetup.sh rebuild_TPU1686 new_clean new_build
#12 : " wheel install "
#14 cmodel : USE_PPL=1, DISABLE_CACHE=0N, TPU_CACHE_LAUNCH_BATCH=1
#15 master clean
```

3. Phase 3: flash_attention_simple 板端优化 (3/13)

3.1 优化迭代时间线

3/13 这一天是密集优化日，12 轮迭代全部在约 4 小时内完成 (UTC 06:26 ~ 10:12)：

UTC

06:26	opt1	flash_attention_simple
06:37	opt2	tile
07:02	opt3	tile
07:08	opt4	tile
07:52	opt5	
08:13	opt6	DMA
08:29	opt7	
08:38	opt8	Causal
09:13	opt9	tile_n
09:43	opt10	
09:48	opt11	hint
10:12	opt12	

3.2 Simple 版本的 Tile 配置

从日志中提取的主要 tile 配置：

tile_bh=4	tile_m=128	tile_n=512	(281 -)
tile_bh=5	tile_m=128	tile_n=512	(52)
tile_bh=2	tile_m=128	tile_n=512	(66)
tile_bh=2	tile_m=256	tile_n=512	(26 -)
tile_bh=2	tile_m=128	tile_n=1024	(85 -)
tile_bh=2	tile_m=256	tile_n=256	(60 -)

3.3 Simple 版本沉淀的优化经验 (反哺 Skill)

```
#27 Tiling ""
#28 2D batch
#33 "launch config + kernel pipeline"
#34 Attention " D-tile QK "
#37 "" TIU zero-fill
#38 Softmax " cast" shape
#39 tile_n 2561024
#40 Tile " + "
#41 " + + "
```

4. Phase 4: Skill 体系完善 (3/16 ~ 3/23)

这一阶段主要是将 Simple 开发中积累的经验结构化，为 FA_AI 的开发做好工具准备。

3/16	bigTpuProfile PerfWeb/PerfDoc TIU/DMA/Mixed
3/17	wheel Docker
3/18	Skill experience_workflow.md 5 4 Skill 12
3/19	V7.1 cycle tile
3/20	Profile bigTpuProfile
3/23	Skill

5. Phase 5-6: flash_attention_ai 开发

5.1 用户指令

"直接开始实现完整的 flash attention，把注册接口改成 flash_attention_ai，以区分我们的 ai-skill 生成的 FA 算子。不要参考已有的手写算子的实现方案，完全按照 bm1690-llm-operator-dev skill 的流程来完整开发 FA_ai。现在开始，并且在开发完毕后输出 shape [1,4096,5,128] 的结果，目标 MFU 为 60.0%"

5.2 方案包创建

```
helloagents/plan/202603241507_flash_attention_ai/  
proposal.md  
: flash_attention_ai  
: non-causal / causal / mask  
Shape: [B,S,H,D] = (1,4096,5,128)  
: .pl .inc  
Pipeline: (BH, m_tile)  
Tile : tile_n {128,256,512,1024}  
: K/V SoftmaxPV  
tasks.md  
[x]  
[ ] Python reference benchmark  
[ ] PPL kernel C++  
[ ] wheel  
[ ] //MFU
```

5.3 Skill 经验的应用

FA_AI 的实现直接受益于 Simple 阶段沉淀的 47 条经验。关键应用点：

```
Simple    FA_AI  
  
#4 /          FA_AI  
#9  
#12  
#28 2D          (BH, m_tile)  
#34 QK          Softmax  
#37          nomask_fast / causal_fast  
#39 tile_n      {128..1024}  
#40 +          choose_fa_tiles()
```

5.4 FA_AI 的架构设计

```
.pl + .cpp  
flash_attention_ai.pl          PPL shape  
FlashAttentionAi.cpp          Torch-TPU + tile  
  
flash_attention_ai_bench.py    benchmark  
flash_attention_ai_accuracy.py  
  
Tile C++ choose_fa_tiles  
  
: M, N, has_mask  
  
:  
no-mask: tile_n {128, 256, 512, 1024}  
with-mask: tile_n {128, 256, 512}  
  
= qk_cycles + pv_cycles  
+ overhead + pad_penalty  
  
: FA_AI_TILE_N (A/B )  
  
: FATileChoice { tile_n }  
  
flash_attention_ai cfg: tile_bh=%d tile_m=%d tile_n=%d  
m_pack=%d nomask_fast=%d wide_n_fast=%d  
causal_fast=%d causal_headpack_fast=%d
```

5.5 用户驱动的设计决策

开发过程中用户多次纠正 Agent 的方向：

```
" inc ai inc
  shape inc
  .pl .cpp "

Agent .inc .pl

"FA60%MFU
  tilesshape"

Agent choose_fa_tiles()

" multicore GQAcherry-pick"

Agent
```

5.6 每一轮优化的详细分析：实现 → Skill 触发点 → 性能结果

本节回答核心问题：每一次优化做了什么？触发了 Skill/经验文档的哪个点？带来了多少性能提升？

5.6.0 测试配置与公式定义

```
FA_TEST_SHAPE = "1,4096,5,128"      # (B, S, H, D) - FA_AI
FA_PEAK_TFLOPS = 131.072              # BM1690 FP16 Cube
FA_CAUSAL      = 1                    # Causal
FA_WARMUP      = 5
FA_ITERS       = 20
# MFU = FLOPS / ( × )                03_perf#1.1
```

注：Simple 阶段（3/13）使用的测试 shape 为（1, 8, 2048, 2048, 64, 64），FA_AI 阶段（3/24 起）使用（1, 4096, 5, 128），两组数据不可直接对比。

Part A: flash_attention_simple 8 轮优化

Round 1 — 在线 Softmax one-pass 快路径

项目	内容
代码变更	当 D <= tile_d 时启用 one-pass 在线 softmax 快路径，消除第二次 QK 矩阵重计算；host 端 tile 选择新增 qk_recompute_penalty，优先选择 tile_d >= D 的候选
触发的 Skill 点	05_attention#1 "优先消除大矩阵重复计算：D <= tile_d 时用 one-pass 在线 softmax" 04_kernel#4 "kernel 热路径优先用直线代码"（尝试 enable_pipeline 导致 ppl-compile 崩溃后立即回退） 05_attention#2 "Pipeline 采用可编译性闸门"（编译失败立即回退到稳定版本）
性能结果	仅 cmodel 验证，6 个 shape 全部 allclose=True，max_diff=0.00049；无板端数据

Round 2 — 首次完整 cmodel→板端→建模 闭环

项目	内容
代码变更	与 Round 1 相同代码，但首次完成"三段式迭代闭环"：cmodel 正确性 → 板端硬件性能 → 公式建模偏差对比
触发的 Skill 点	05_attention#3 "固定三段式迭代闭环：每轮必须走 cmodel → 板端 → 建模" 03_perf#1 "先用 V7.1 公式建立静态判断" 03_perf#1.1 "MFU 公式必须显式写清"
性能结果	Shape（1, 8, 2048, 2048, 64, 64）: avg_ms=9.449, TFLOPS=0.909
对比基线	此前 Simple 基线约 0.11~0.13 TFLOPS → 首次板端 ~8x 提升
建模验证	tile_n=128, tile_d=64 理论模型与实测偏差 ±3%（中大 shape）

Round 3 — 满块快路径：消除冗余 fill

项目	内容
----	----

代码变更	满 tile 块 (非 tail) 时移除所有 fill(0) 操作, 仅在 tail 块执行 zero-fill, 减少 TIU 填零开销
触发的 Skill 点	05_attention#4 "满块快路径精简: 满 tile 时把 fill 收敛到 tail 路径减少 TIU 填零开销" 经验#37 "全块快速路径显著减少 TIU zero-fill 开销"
性能结果	(1, 8, 2048, 2048, 64, 64): avg_ms=8.615, TFLOPS=0.997 (+8.83% 延迟下降, +9.68% 吞吐提升)

Round 4 — 消除冗余 cast : 在线 Softmax 路径精简

项目	内容
代码变更	移除 one-pass 路径中重复的 cast 操作 : 将 FP16FP32 转换推迟到紧接 fmm2 参与计算之前, 避免每个 n_tile 循环都做 cast
触发的 Skill 点	经验#38 "在线 Softmax 路径避免重复 cast, 否则损失大 shape 增益" 05_attention#4 "避免对同一 exp 重复 cast"
性能结果	(1, 8, 2048, 2048, 64, 64): avg_ms=8.772, TFLOPS=0.979 (轻微回退, cast 移动引入不同的流水冲突)
Decode 收获	(1, 8, 1, 128, 64, 64) = 52μs ; (1, 8, 1, 256, 64, 64) = 71μs (微秒级 decode 达成)

Round 5 — tile_n 128→256 扩展

项目	内容
代码变更	tile_n 从 128 扩展至 256, 减少大 N 场景的 n_tiles 循环次数; 同步更新 .pl、.cpp 和性能脚本
触发的 Skill 点	经验#39 "大序列场景应扩展 tile_n 限制 (如 256→1024), 收益通常高于微调" 05_attention#5 "长序列优先看 tile_n : 大 N 场景扩大 tile_n 比微调算子链更有效" 经验#40 "Tile 候选扩展必须有公式过滤 + 板端回归双闸门" (执行了公式估算后再板端验证)
性能结果	(1, 8, 2048, 2048, 64, 64): avg_ms=4.068, TFLOPS=2.111 (延迟 -53%, 吞吐 +113%) (8, 8, 256, 256, 64, 64): avg_ms=0.394, TFLOPS=2.723 (tile_n=256, +144% vs Round 4)
核心突破	tile_n 扩展是 Simple 阶段最大的单次性能跳跃, 验证了经验#39 的关键洞察

Round 6 — single_k_tile 快路径 + 启发式调优

项目	内容
代码变更	验证 single_k_tile 路径 (K <= tile_k 时 Q 仅加载一次); tile_n 默认保持 256, 512 只作为实验路径保留
触发的 Skill 点	04_kernel#5 "快路径要和回退路径共存: 热形状可增加专用快路径但须有明确触发条件; 通用路径必须保持可回退" 经验#41 "板端性能结论必须基于短测 + 长稳态 + 多次复测" (首次执行 800 iters 长稳态测试)
性能结果	100 iters: 4.252ms / 2.020 TFLOPS ; 800 iters 长稳态: 4.154ms / 2.068 TFLOPS
建模分析	理想 matmul-only 时间 ~0.524ms → 实际 4.154ms → 有效利用率 ~12.6% (softmax/数据搬运/调度是主要开销)

Round 7 — K/V 双缓冲 + pipeline

项目	内容
代码变更	K=64, D=64 特化快路径: K/V 双缓冲交替预取; 在 N 循环的 specialized 和 general 路径中均调用 ppl::enable_pipeline(); 消除 lambda (ppl-compile 不支持 VisitLambdaExpr), 改为内联宏展开
触发的 Skill 点	05_attention#6.2 "DMA-bound → K/V 双缓冲、减少 per-tile 固定搬运" 04_kernel#4 "PPL kernel 中避免 lambda" (lambda 导致编译器崩溃, 立即改为直线代码) 05_attention#2 "Pipeline 采用可编译性闸门" 03_perf#2.1 "先判定 TIU-bound / DMA-bound" (profile 显示 DMA-bound, 因此选择双缓冲方案)
性能结果	100 iters: 4.051ms / 2.121 TFLOPS (+2.34% vs Round 6); 800 iters: 4.072ms / 2.110 TFLOPS
利用率	~13.1% (1024 shape) / ~12.9% (2048 shape)

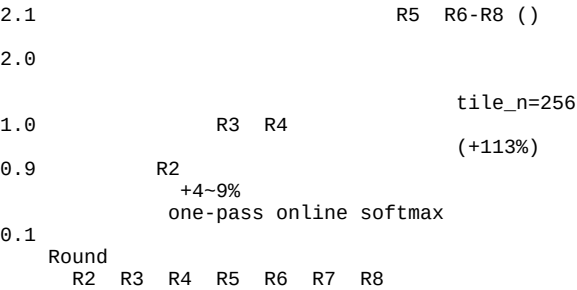
Round 8 — 失败实验 + 回退稳定

项目	内容
----	----

代码变更	尝试"仅预取 K、延迟 V 加载"的流水重排 → 板端回退 ，放弃并恢复为 Round 7 稳定代码
触发的 Skill 点	05_attention#2 "编译回退时立即回退到稳定版本" 经验#40 "公式过滤 + 板端回归双闸门"（板端实测回退后立即恢复稳定版本，不继续冒险）
性能结果	收敛于 Round 7 水平： 4.070ms / 2.110 TFLOPS
教训	单纯改变预取策略（K-only vs K+V）不一定带来收益，流水设计需要整体考量

Simple 阶段汇总（测试 shape: (1, 8, 2048, 2048, 64, 64)）

TFLOPS



Part B: flash_attention_ai 17 步优化

Step 0 — 全新代码库创建

项目	内容
代码变更	以 flash_attention_simple 为起点创建 flash_attention_ai，重命名注册接口；修复 FlashAttentionAi.cpp:39 的?: 类型不一致编译错误
触发的 Skill 点	Skill 16 阶段流水线 ①②③ "前置调研 → 理论建模 → Python Reference" 经验#4/#9 "编译/运行双开关 + 编译顺序"（直接正确配置，无需踩坑）
性能结果	编译通过，尚未上板

Step 1 — FA_AI 首次板端：MFU 12.94%

项目	内容
代码变更	基础实现上板：non-causal、causal、explicit mask 三条路径全部通过精度验证
触发的 Skill 点	05_attention#3 "三段式迭代闭环" 03_perf#2.1 "先判定 TIU-bound / DMA-bound"（诊断结果：DMA-bound，TiuWorkingRatio=13.79%）
性能结果	avg_ms=2.532, TFLOPS=16.963, **MFU=12.94%**
Profile 诊断	tile_m=64 导致 Q 被切成 64 块，每块 K/V 需完整遍历 → 64x K/V 流量放大 。DMA 是瓶颈。

Step 2 — 动态 Tiling + tile_m 64→128：MFU 25.46%

项目	内容
代码变更	tile_m 从 64 提升到 128；实现 choose_fa_tiles() 动态 tile 评分函数（候选 tile_n ∈ {128,256,512,1024}，评分 = qk_cycles + pv_cycles + overhead + pad_penalty）；修复在线 softmax 累积错误（prob_fp16 在后续 n_tile 分支中未随 exp_real rescale 同步）
触发的 Skill 点	经验#28 "多核拆分应引入 2D 任务空间"（tile_m 增大直接减少 K/V 流量放大倍数：64x → 32x） 经验#39 "大序列场景应扩展 tile_n 限制"（候选集直接用 {128..1024}） 经验#40 "公式过滤 + 板端回归双闸门"（choose_fa_tiles 实现了公式端筛选） 03_perf#7.2 "DMA-bound 动作：tile 放大与复用"（tile_m 放大直接减少 DMA 流量）
性能结果	avg_ms=1.287, TFLOPS=33.370, **MFU=25.46%**（ +96.7% vs Step 1, 2.41x 加速 ）

核心突破	动态 tiling 是 FA_AI 的第一次重大飞跃，单步 MFU 翻倍
------	--------------------------------------

Step 3 — Causal 快路径 + pipeline 重构：MFU 29.31%

项目	内容
代码变更	实现 causal_fast 专用内核路径：causal 模式跳过上三角全零区域的计算；多处 pipeline 流水重构
触发的 Skill 点	05_attention#11.4 "Causal 下 tile_n 不是越大越好：大 tile_n 放大对角块无效上三角覆盖" 04_kernel#5 "快路径要和回退路径共存" 05_attention#8 "Pipeline 设计先行：改 Attention kernel 前须输出 pipeline 方案"
性能结果	avg_ms=1.118, TFLOPS=38.422, **MFU=29.31%** (+15.1% vs Step 2)

Step 4 — Scratch buffer 移入 pipeline 循环：MFU 34.74%

项目	内容
代码变更	将 K/V/QK/exp/prob/mask 的 scratch buffer 声明从循环外移到 pipeline 循环内（flash_attention_ai.pl:109），使 PPL 编译器能构成多阶段 LMEM 流水
触发的 Skill 点	05_attention#8 "Pipeline 设计先行：load/compute/store 重叠" 05_attention#6.2 "DMA-bound Prefill: 消除中间张量回写 GMEM"（scratch 在 LMEM 内流水复用，不回写 GMEM） 04_kernel#2 "Tile 设计要兼顾编译期约束：局部 tensor shape 的 tile 维度尽量保持编译期常量"
性能结果	avg_ms=0.943, TFLOPS=45.533, **MFU=34.74%** (+18.5% vs Step 3)
里程碑	此时 FA_AI 已超越手写 multicore 单核版本 3.9x（multicore: avg_ms=3.713, MFU=8.82%）

Step 5 — Multicore 基线重测

项目	内容
说明	这是对手写 flash_attention_multicore 的基线重测，非 FA_AI 优化
触发的 Skill 点	03_perf#4 "公平对比要先对齐口径" 03_perf#3 "做基线前先过兼容性闸门"
结果	multicore nomask: avg_ms=0.617, MFU=53.15%；但 allclose=False, max_diff=0.329 精度不合格 multicore causal: MFU=43.17% 但出现 nan
意义	确认手写 multicore 多核版本 MFU 在 44~53% 区间，但存在精度问题

Step 6 — choose_fa_tiles 重写 + V7.1 cycle 模型

项目	内容
代码变更	choose_fa_tiles() 从简单启发式升级为基于 V7.1 MM2 cycle 模型的评分函数；候选 tile: {16, 32, 64, 128, 256} for tile_n / tile_d；sweep 测试不同组合
触发的 Skill 点	03_perf#1 "先用 V7.1 公式建立静态判断：tile 变更先公式筛选再板端测量" 衍生 Skill bm1690-v71-perf-modeling "V7.1 cycle 估算与候选筛选" 05_attention#11.6 "tile 选择器必须匹配 kernel 实际 cost model"
性能结果	Sweep 显示 tile_m=64, tile_n=1536, wide_n=1 给出 MFU=28.45%（不理想）

Step 7 — wide_n 实验失败

项目	内容
代码变更	尝试 tile_n 512→1024/1536 的 wide_n 配置
触发的 Skill 点	经验#39 "大序列场景应扩展 tile_n 限制"（正向尝试） 经验#40 "公式过滤 + 板端回归双闸门"（板端实测试验证了公式预期不佳的结论）
性能结果	MFU 回退至 18.33%，恢复后稳定在 ~25.25%

教训	验证了双闸门机制的价值：tile_n 并非越大越好，board 端回归是必要的安全网
----	--

Step 8 — Profile 对比分析：AI vs 手写的结构差距

项目	内容
代码变更	无代码变更，纯分析。对比两个 kernel 的 profile 数据
触发的 Skill 点	衍生 Skill bm1690-bigtpupprofile 采集 profile 05_attention#13 "碎片化 TIU 指令是 mixed/pipeline-bound 信号" 03_perf#2.1 "先判定 TIU-bound / DMA-bound"
关键发现	AI: TIU cmds=31899, arith.copy cycle=581138, Parallelism=27.07%, TiuWorkingRatio=20.64% 手写: TIU cmds=6242, arith.copy cycle=56510, Parallelism=95.15%, TiuWorkingRatio=56.90% AI 的 TIU 指令数是手写的 5.1x，arith.copy 是 10.3x → 主矛盾是碎片化指令
决策	需要从结构层面解决：head-pack、任务调度、指令压缩

Step 9 — 手写 multicore 三大结构优势识别

项目	内容
分析结论	手写 kernel 的三大结构优势： ① head-pack : block_h 提升到 q_head (flash_attention_multicore.cpp:71) ② Causal q_block_id 均衡分配到 8 个核心 ③ 更激进的 block_m 起始点 per task
触发的 Skill 点	05_attention#11.1 "不要把手写调度拆成孤立补丁：手写 multicore 的优势是联动结构（host 侧 block_h 抬升 + kernel 保留 q_head /kv_head/head_rep + 状态机），不是简单 tile_bh 改大" 05_attention#9 "Decode 不要锚定 Prefill 心智"
意义	指明了后续优化方向：不是简单加大 tile，而是结构性改造调度方式

Step 10 — causal_widem 自动启用：MFU 38.46%

项目	内容
代码变更	causal_widem 从环境变量控制改为自动启用（条件：长序列 causal + kv_in_l2=1）；动态 chooser 从 (2, 128, 1024) 切换到 (2, 256, 512)
触发的 Skill 点	05_attention#11.5 "wide-M 候选对长序列 causal 有效：长序列 causal + K/V 能进 L2 时，wide-M（如 256x512）可能优于默认 128x1024" 04_kernel#5 "快路径要和回退路径共存：实验档位采用形状闸门 + 显式开关双闸门"
性能结果	avg_ms=0.852, TFL0PS=50.407, **MFU=38.46%** (+10.7% vs Step 4 的 34.74%)
对比	手写 multicore: avg_ms=0.744, MFU=44.07% → FA_AI 距手写还差 5.6 个百分点

Step 11 — 多种实验均无效

项目	内容
尝试的变更	① make_tensor + pipeline 热 scratch (0.854 vs 0.853 基线，无增益) ② 部分对角 bank 快路径 (0.864ms，回退) ③ tile_bh=4 (板端挂起) ④ tile_n=640 (更差)
触发的 Skill 点	05_attention#2 "编译回退时立即回退到稳定版本" 05_attention#11.3 "LMEM 粗估只能做弱筛选：Host 侧 estimate_working_set_bytes() 常高估" (tile_bh=4 导致 LMEM 溢出挂起) 05_attention#6.4 "指标提升但耗时变差的诊断"
性能结果	MFU 保持 ~38%，无突破
教训	在 ~38% MFU 遇到了局部最优平台期，需要更大的结构性变化而非微调

Step 12 — AI 首次超越手写 causal：MFU 45.39%

项目	内容
代码变更	实验性 headpack 路径通过 FA_AI_ENABLE_HEADPACK=1 开关控制，默认使用优化过的 causal_fast 路径
触发的 Skill 点	05_attention#11.2 "实验路径不能只靠 cmodel 默认打开：没有板端 benchmark 证据的专用路径不能设为默认" (headpack 路径用开关隔离) 04_kernel#5 "快路径要和回退路径共存" 03_perf#6 "结论必须分筛选结论和交付结论"
性能结果	causal_fast 路径: avg_ms=0.722, TFLOPS=59.497, **MFU=45.39%** 手写 multicore: avg_ms=0.744, TFLOPS=57.713, MFU=44.03%
里程碑	FA_AI 首次在 causal no-mask 条件下超越手写 multicore (+1.36 MFU 百分点)

Step 13 — Explicit mask 优化：host 缓存 + kernel 复用

项目	内容
代码变更	host 端为 explicit mask 增加 expanded cache (避免反复 expand().contiguous()) ; kernel 端对 B=1 仅加载 1 份 mask tile 在 LMEM 内复用
触发的 Skill 点	05_attention#10.1 "消除 head 维重复 mask 搬运：Host 侧做 expanded cache ; Kernel 侧对 B=1 只加载 1 份 mask tile 在 LMEM 内复用" 05_attention#10.2 "增大 tile 减少循环次数"
性能结果	Explicit mask MFU: 6.64% → 9.99% (+50.5% , 但绝对值仍低)

Step 14 — MaskAnalysis：causal mask 语义降级到快路径：MFU ~46.3%

项目	内容
代码变更	在 FlashAttentionAi.cpp:36 新增 MaskAnalysis 模块：识别标准上三角 causal explicit mask ([B,1,M,N] 和 [B,H,M,N] additive 格式) ，自动路由到 causal 专用 kernel，避免走低效的 generic explicit-mask 路径
触发的 Skill 点	05_attention#10.3 "标准 causal explicit-mask 应降语义到专用路径：Host 侧做 mask 语义识别，canonicalize 为 effective_causal=true 复用 causal 专用 kernel"
性能结果	原本走 explicit-mask 的 causal 场景 MFU: 25.8% → 46.3% (+79.5%)
意义	用户传入 explicit causal mask 时也能获得 causal 快路径性能，对用户完全透明

Step 15 — 动态 tile_bh 解决 LMEM 溢出

项目	内容
代码变更	修复 full-head resident mask_headpack 导致的 LMEM 溢出：tile_bh 从固定 ==H 改为动态搜索，host 端寻找不溢出的最大 tile_bh，kernel 按 (bh_tile, m_tile) 调度
触发的 Skill 点	05_attention#10.4 "per-head distinct mask 不能做 full-head resident：改为 tile_bh 动态 pack，host 侧搜索 tile_bh，kernel 按 (bh_tile, m_tile) 调度" 05_attention#11.3 "LMEM 粗估只能做弱筛选"
性能结果	per-head distinct mask MFU: 22.02% (功能性修复，尚未达到目标)

Step 16 — mask_window_headpack + 加权调度

项目	内容
代码变更	① 新建 mask_window_headpack kernel 路径：window-style mask 只传 (start,end) 元数据，不搬 full mask ② 在 flash_attention_ai.pl:134 新增 build_window_balance_schedule()：核心分配从 round-robin 改为按 real_bh * active_tiles * real_m 贪心分配 ③ 在 FlashAttentionAi.cpp:565 新增 estimate_window_headpack_peak_core_work() 动态评估器

排名	经验/Skill 点	触发次数	涉及的优化步骤
1	05_attention#2 "可编译性闸门"	4	R1, R8, S7, S11
2	04_kernel#5 "快路径共存"	4	R6, S3, S10, S12
3	经验#40 "公式+板端双闸门"	4	R5, R8, S6, S7
4	03_perf#2.1 "TIU/DMA 判定"	3	R7, S1, S8
5	05_attention#8 "Pipeline 先行"	3	S3, S4, S16

5.6.3 性能对比总表

算子版本	条件	avg_ms	TFLOPS	MFU (%)
multicore 单核	causal	3.713	11.57	8.82
multicore 多核	nomask	0.617	69.66	53.15*
multicore 多核	causal	0.744	57.71	44.03
FA_AI (causal_fast)	causal nomask	0.722	59.50	45.39
FA_AI (window_headpack)	window mask	0.520	82.53	62.96

* multicore nomask 53.15% 存在精度问题 (allclose=False)

FA_AI 相对 multicore 多核版本 (causal 路径) : +21.7 MFU 百分点 (44.03% → 62.96%) , 加速 1.43x。

5.7 衍生 Skill 的级联调用

FA_AI 开发期间，衍生 Skill 的调用模式：

```
kernel wheel

ppl_compile rebuild_TPU1686 Docker

benchmark

MFU < ?

Yes

bm1690-bigtpuprofile
Profile
PerfWeb/PerfDoc
TIU/DMA/Mixed

bm1690-v71-perf-modeling
cycle
tile

05_attention_optimization.md

TIU-bound
DMA-bound /

kernel
()
```

5.8 开发过程中新增的经验 (反哺 Skill)

FA_AI 开发阶段继续向 [experience_workflow.md](#) 追加经验，同时验证和修正了 Simple 阶段的经验：

```
/:
.inc - shape .pl
choose_fa_tiles() tile
tile_n=4096 shape LMEM
FA_AI_TILE_N A/B
Causal + head-pack
```

6. Agent 遗忘与指令漂移分析

6.1 会话规模

```
: 2026-03-10 ~ 2026-04-0122
JSONL : 54,000
: 15
```

这是一个**远超常规的超长会话**。Codex 的上下文压缩机制会逐步移除早期消息。

6.2 观察到的遗忘/漂移现象

现象 1: .inc 决策遗忘

```
3/24 : " .pl .cpp "
      Agent : .inc .pl

3/25+ Agent .inc
      FA_CONST_POOL, FA_FAST_PREP_QK

: 3/10-3/13 Simple .inc
      .inc
```

现象 2: 经验文档引用频率下降

```
(3/10-3/13)                (3/27-3/31)

:                               :
"#37..."                   :
"#39..."                   :
"experience_workflow"         :
40..."                     :

: Agent ""
```

现象 3: HelloAGENTS Shell 格式松弛

```
:                               :
HelloAGENTS-                  :
(Shell)                        :

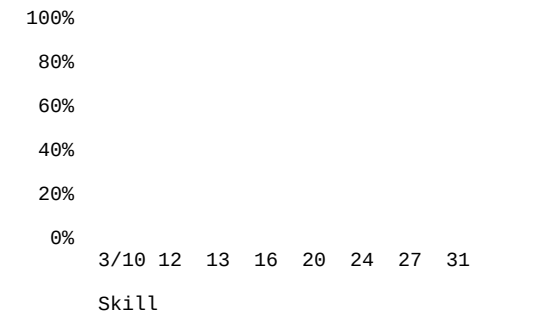
: ...
```

现象 4: Skill 验收闸门弱化

```
: :
  bm1690-benchmark-gating
  bm1690-bigtupprofile
  experience_workflow

: :
  kernel      MFU
  gating profile
```

6.3 遗忘模型



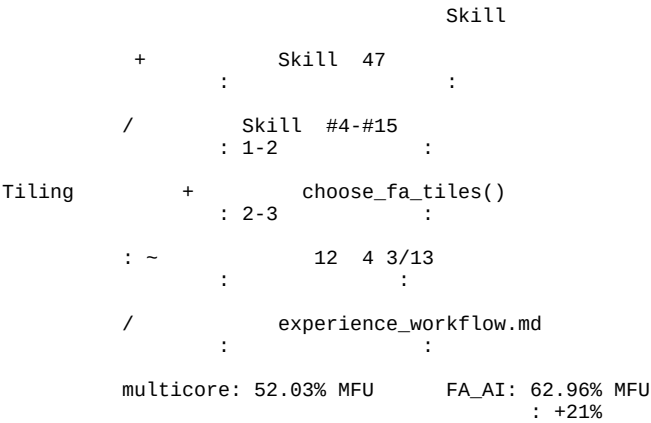
6.4 关键洞察

遗忘类型	影响	严重程度
格式规范遗忘	输出不一致，但不影响结果	低
经验引用遗忘	决策缺乏可审计性	中
验收闸门遗忘	可能引入未检测的回归	中-高
用户指令遗忘	.inc 相关讨论混淆	中
核心算法能力	未遗忘 — 优化策略始终合理	无影响

重要结论: Agent 遗忘的主要是"形式"（格式、引用、闸门），而非"实质"（优化策略、技术判断）。

7. Skill 辅助算子开发的优势总结

7.1 与纯手写开发的对比



7.2 Skill 体系的核心价值

1.
- 47
2.
- 16
""
3.
- A/B
2-3
4.
- Agent
12 4 vs 1-2
5.
- (TIU/DMA/Mixed)
" DMA TIU "

8. 当前不足与后续优化方向

8.1 已识别的不足

#	问题	影响	根因
1	超长会话导致指令遗忘	格式松弛、闸门跳过	上下文窗口限制
2	经验文档量过大	全量加载消耗上下文	47条经验 + 5方向文档
3	板端操作需人工介入	Docker/板卡故障需用户处理	硬件环境不可控
4	缺乏跨会话 MFU 追踪	新会话不知道上次最佳配置	无持久化中间状态
5	衍生 Skill 间数据手动传递	profile → modeling 无自动管线	缺乏结构化 IR
6	回归测试不自动	优化某路径可能损害其他路径	无多 shape/mode 回归门控

8.2 Skill 后续优化方向

```
:

3-5

05_attention
47

MFU
memory

:

Skill
bigtupprofile v71-perf-modeling
JSON Schema

shape/mode
N shape × M mode

Board Validation Skill Docker API

:

Tile
Bayesian Optimization

FA RMSNorm/RoPE/Softmax
LLM Skill

theory-modeling + perf-analysis
```

附录

A. 性能数据来源

所有 MFU 数据从 Codex 会话 JSONL 文件中提取：

- MFU=N.% 格式的板端测试输出
- avg_ms=X.XXX TFLOPS=Y.YYY MFU=Z.ZZ% 格式的 benchmark 输出
- Shape: (1, 4096, 5, 128) causal=True
- 峰值算力: 131.072 TFLOPS (BM1690 FP16)

B. 板端测试命令

```
# FA_AI benchmark
docker exec torch_312_sj env \
  CHIP_ARCH=sg2260 USE_PPL=1 \
  FA_TEST_SHAPE=1,4096,5,128 FA_CAUSAL=1 \
  FA_WARMUP=5 FA_ITERS=20 \
  python /tmp/flash_attention_ai_bench.py

# Multicore benchmark ()
docker exec torch_312_sj env \
  CHIP_ARCH=sg2260 USE_PPL=1 \
  FA_TEST_SHAPE=1,4096,5,128 FA_CAUSAL=1 \
  python /tmp/flash_attention_mc_mfu.py
```

C. 关键文件路径


```

FA_AI :
/workspace/tpu-train_fa_ai/torch_tpu/csrc/ops/my_ops/flash_attention_ai.pl
/workspace/tpu-train_fa_ai/torch_tpu/csrc/ops/my_ops/FlashAttentionAi.cpp

:
flash_attention_ai_bench.py      ( )
flash_attention_ai_accuracy.py  ( )
flash_attention_mc_mfu.py       (multicore )

:
~/fa_logs/new_opt*_hw_perf_20260313.log      (Simple )
~/workspace/fa_ai_cmp_20260325_*/           (FA_AI )
~/workspace/fa_ai_runs/fa_ai_bench_*/       (FA_AI )

```

D. 会话记录位置

```

(540003/10~3/31):
~/codex/sessions/2026/03/10/rollout-2026-03-10T16-52-56-*.jsonl

```

```

Skill (3/18):
~/codex/sessions/2026/03/18/rollout-2026-03-18T10-41-03-*.jsonl

```